

Lecture7a

October 27, 2025

1 CS5489 - Machine Learning

2 Lecture 7a - Deep Learning

2.0.1 Dept. of Computer Science, City University of Hong Kong

3 Outline

- Going deeper
 - ReLU and Batchnorm
- Optimization methods
- Deep architectures and Image classification
- Transfer learning

```
[1]: # setup
%matplotlib inline
import matplotlib_inline # setup output image format
matplotlib_inline.backend_inline.set_matplotlib_formats('retina')
import matplotlib.pyplot as plt
plt.rcParams['figure.dpi'] = 100 # display larger images
import matplotlib
from numpy import *
import numpy as np
from sklearn import *
from scipy import stats
from scipy import signal
import random
import math
from prettytable import PrettyTable

rbow = plt.get_cmap('rainbow')
```

```
[2]: import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
import struct
import sys
```

```

print(f"Python: {sys.version} PyTorch: {torch.__version__}")

# Set device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

```

Python: 3.13.5 | packaged by Anaconda, Inc. | (main, Jun 12 2025, 16:09:02) [GCC 11.2.0] PyTorch: 2.8.0+cu129
Using device: cuda

```

[3]: def plot_history(history):
    fig, ax1 = plt.subplots()

    ax1.plot(history['train_loss'], 'r', label="training loss ({:.6f})".
    ↪format(history['train_loss'][-1]))
    ax1.plot(history['val_loss'], 'r--', label="validation loss ({:.6f})".
    ↪format(history['val_loss'][-1]))
    ax1.grid(True)
    ax1.set_xlabel('iteration')
    ax1.legend(loc="best", fontsize=9)
    ax1.set_ylabel('loss', color='r')
    ax1.tick_params('y', colors='r')

    if 'train_accuracy' in history:
        ax2 = ax1.twinx()

        ax2.plot(history['train_accuracy'], 'b', label="training acc ({:.4f})".
        ↪format(history['train_accuracy'][-1]))
        ax2.plot(history['val_accuracy'], 'b--', label="validation acc ({:.
        ↪4f})".format(history['val_accuracy'][-1]))

        ax2.legend(loc="best", fontsize=9)
        ax2.set_ylabel('acc', color='b')
        ax2.tick_params('y', colors='b')
    plt.show()

```

```

[4]: def show_imgs(W_list, nc=10, highlight_green=None, highlight_red=None,
    ↪titles=None):
    nfilter = len(W_list)
    nr = (nfilter - 1) // nc + 1
    for i in range(nr):
        for j in range(nc):
            idx = i * nc + j
            if idx == nfilter:
                break
            plt.subplot(nr, nc, idx + 1)
            cur_W = W_list[idx]

```

```

plt.imshow(cur_W, cmap='gray', interpolation='nearest')
if titles is not None:
    plt.title(titles % idx)

if ((highlight_green is not None) and highlight_green[idx]) or \
    ((highlight_red is not None) and highlight_red[idx]):
    ax = plt.gca()
    if highlight_green[idx]:
        mycol = '#00FF00'
    else:
        mycol = 'r'
    for S in ['bottom', 'top', 'right', 'left']:
        ax.spines[S].set_color(mycol)
        ax.spines[S].set_lw(2.0)
    ax.xaxis.set_ticks_position('none')
    ax.yaxis.set_ticks_position('none')
    ax.set_xticks([])
    ax.set_yticks([])
else:
    plt.gca().set_axis_off()

```

```

[5]: def read_32int(f):
    return struct.unpack('>I', f.read(4))[0]
def read_img(img_path):
    with open(img_path, 'rb') as f:
        magic_num = read_32int(f)
        num_image = read_32int(f)
        n_row = read_32int(f)
        n_col = read_32int(f)
        #print 'num_image = {}; n_row = {}; n_col = {}'.format(num_image,
↪n_row, n_col)
        res = []
        npixel = n_row * n_col
        res_arr = fromfile(f, dtype='B')
        res_arr = res_arr.reshape((num_image, n_row, n_col), order='C')
        #print 'image data shape = {}'.format(res_arr.shape)
        return num_image, n_row, n_col, res_arr
def read_label(label_path):
    with open(label_path, 'rb') as f:
        magic_num = read_32int(f)
        num_label = read_32int(f)
        #print 'num_label = {}'.format(num_label)
        res_arr = fromfile(f, dtype='B')
        #print res_arr.shape
        #res_arr = res_arr.reshape((num_label, 1))
        res_arr = res_arr.ravel()
        #print 'label data shape = {}'.format(res_arr.shape)

```

```
return num_label, res_arr
```

```
[6]: n_train, nrow, ncol, training = read_img('data/train-images.idx3-ubyte')
_, trainY = read_label('data/train-labels.idx1-ubyte')
n_test, _, _, testing = read_img('data/t10k-images.idx3-ubyte')
_, testY = read_label('data/t10k-labels.idx1-ubyte')

# for demonstration we only use 10% of the training data
sample_index = range(0, training.shape[0], 10)
training = training[sample_index]
trainY = trainY[sample_index]
print(training.shape)
print(trainY.shape)
print(testing.shape)
print(testY.shape)
```

```
(6000, 28, 28)
(6000,)
(10000, 28, 28)
(10000,)
```

```
[7]: # Convert to PyTorch tensors and scale to 0-1
trainI = torch.from_numpy(training.reshape((6000, 1, 28, 28)) / 255.0).float()
testI = torch.from_numpy(testing.reshape((10000, 1, 28, 28)) / 255.0).float()

print(trainI.shape)
print(testI.shape)
```

```
torch.Size([6000, 1, 28, 28])
torch.Size([10000, 1, 28, 28])
```

```
[8]: # Generate fixed validation set
from sklearn.model_selection import train_test_split

vtrainI_np, validI_np, vtrainY_np, validY_np = train_test_split(
    trainI.numpy(), trainY,
    train_size=0.9, test_size=0.1, random_state=4487
)

vtrainI = torch.from_numpy(vtrainI_np).float()
validI = torch.from_numpy(validI_np).float()
vtrainY = torch.from_numpy(vtrainY_np).long()
validY = torch.from_numpy(validY_np).long()

validsetI = (validI, validY)

print(f"Training set: {vtrainI.shape}, {vtrainY.shape}")
```

```
print(f"Validation set: {validI.shape}, {validY.shape}")

cid, cfreq = vtrainY.unique(return_counts=True)

print(f"Training set Distribution:", cfreq.numpy().tolist())
```

```
Training set: torch.Size([5400, 1, 28, 28]), torch.Size([5400])
Validation set: torch.Size([600, 1, 28, 28]), torch.Size([600])
Training set Distribution: [491, 599, 554, 556, 513, 518, 567, 584, 496, 522]
```

4 Problems with Going Deeper

- **Vanishing Gradient Problem 1**
 - successive multiplications of small gradients gives smaller gradients, and converges to 0
 - the gradients backpropagated to the first few layers has small signal.
- Example: for 4-layers,

$$\frac{dL}{da_j} = \frac{dg_1^T}{da_j} \frac{dz_1^T}{dg_1} \frac{dg_2^T}{dz_1} \frac{dz_2^T}{dg_2} \frac{dg_3^T}{dz_2} \frac{dz_3^T}{dg_3} \frac{dg_4^T}{dz_3} \frac{df^T}{dg_4} \frac{dL}{df}$$

- **Vanishing Gradient Problem 2**
 - using backprop, the gradient at a node is the summation over $O(H^D)$ paths
 - * D is the number of layers to the output layer.
 - * H is the number of nodes in the layer (assuming they all have the same width).
 - the original loss signal gets “washed out”.
- **Dataset Size**
 - a “small” network with just 40 inputs, 30 hidden nodes, and 1 output has ~1200 parameters.
 - if we don’t have enough samples:
 - * large variance in the parameter estimator (what you get may be far from the truth)
 - * deeper networks are more complex, which are easier to overfit the training data.
- *How many samples do we need?*
 - Theorem (Bartlett, Maierov, Meir, 1998) > Suppose N is a feed-forward network with W weights, L layers, and all non-output gates having a fixed piecewise-polynomial activation function with a fixed number of pieces (e.g., ReLU). Then $VCdim(N) = O(WL \log W + WL^2)$.
- VC dimension is a statistical measure of function class capacity:
 - to achieve certain level of generalization, sample size needs to scale proportionally to VC dim.
 - for the same number of parameters, the deeper network requires more data (WL^2).
 - increasing the number of weights, requires a super-linear increase in sample size ($W \log W$).

5 ReLU activation function

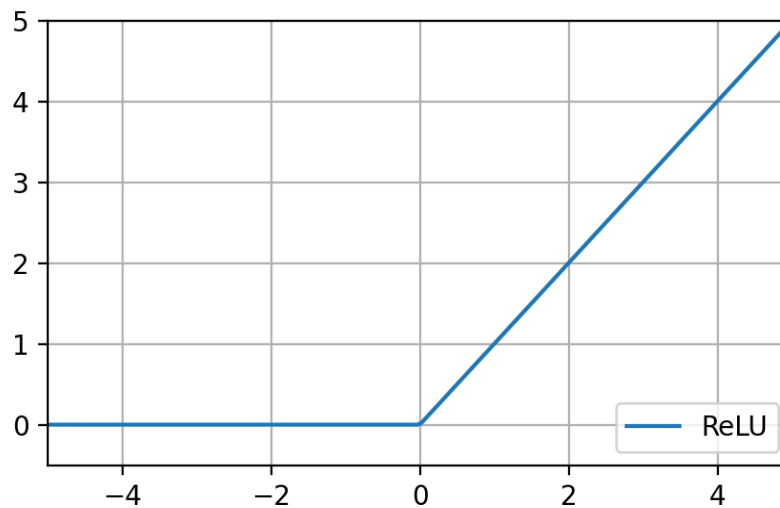
- Rectified Linear Unit: $\text{ReLU}(z) = \max(0, z)$
 - easier to train with: gradient is either 0 or 1.
 - faster: don't need to calculate exponential
 - sparse representation: most nodes will output zero.

```
[9]: x=linspace(-5,5,200)
fs = {'ReLU': lambda x: maximum(0,x),
      }

actfig = plt.figure(figsize=(5,3))
for name,f in fs.items():
    plt.plot(x, f(x), label=name)
plt.legend(loc="lower right")
plt.axis([-5,5,-0.5,5])
plt.grid(True)
plt.close()
```

```
[10]: actfig
```

```
[10]:
```



6 Advantage of Sparsity

- if a hidden node $h = 0$, then $\frac{dL}{dh} = 0$.
 - This blocks some paths when computing the gradients.
 - Gradient signal is less washed out.
 - Reduces the vanishing gradient problem.

7 Better Network Parameterization

- There are equivalent parameterizations of the network by scaling up/down the weights in successive layers.
 - $f(\mathbf{x}) = \mathbf{A}^T r(\mathbf{B}^T \mathbf{x}) = \frac{1}{\epsilon} \mathbf{A}^T r(\epsilon \mathbf{B}^T \mathbf{x})$
 - * $\epsilon > 0$ and $r(\cdot)$ is the ReLU activation.
- **Problem:**
 - “internal covariate shift” - change in the distribution of activations during training, due to changes in the parameters.

8 Why is it bad?

- suppose, we have a linear network:
- $y = xw_1w_2w_3w_4$
- gradient of each layer is $(g_1, \dots, g_4)$
- update the parameters with GD:
 - $y = x(w_1 - \eta g_1)(w_2 - \eta g_2)(w_3 - \eta g_3)(w_4 - \eta g_4)$
 - * there are many higher-order terms, e.g., $w_1w_2\eta^2g_3g_4$
 - although w_i are updated independently, they strongly affect each other.
 - hence, if the distribution of activations changes in 1 layer, then all layers are affected, and we need to adjust other layers.

9 Solution: Batch Normalization

- For each node in each layer, normalize the outputs to zero mean and unit variance, *over each mini-batch*.
 - this is analogous to the idea of normalizing the input feature vector to (0,1) Gaussian with standard ML models!
- Place batchnorm layer after linear transformation.
 - Let $\{y_i\}_{i=1}^N$ be the output of the linear transform in one minibatch.
 - For each node (dimension) in the layer:
 - * normalize: $\hat{y}_i = \frac{y_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$
 - μ, σ^2 are the mean and variance of $\{y_i\}$ in the mini-batch.
 - ϵ is a small constant for numerical stability.
 - * scale-shift: $z_i = \gamma \hat{y}_i + \beta$
 - γ, β are learnable parameters
 - puts the output in the proper regime of the non-linear activation.
 - * The final distribution has mean β and variance γ^2 .
- Notes:
 - batchnorm is applied to each node independently.
 - should put the batchnorm layer after the linear transformation layer.
 - the bias of the linear layer is not necessary since it is removed by batchnorm
- Training:
 - gradients can be computed through the batchnorm layer as usual.
- Training effects:

- training is accelerated; can use higher learning rates
- more stable gradients during training
 - * increasing the scale of the activations decreases the gradient
 - * self-correcting stabilization.
- better generalization
 - * no need for dropout or L2 regularization.

10 Example: MNIST

- for each Conv2D/Dense layer:
 - change activation to linear (default); remove bias term
 - append batch-norm and ReLU activation

```
[11]: class DeepCNN(nn.Module):
    def __init__(self, weight_decay=0.0001):
        super(DeepCNN, self).__init__()
        self.weight_decay = weight_decay

        self.conv1 = nn.Conv2d(1, 10, kernel_size=5, stride=2, padding=2)
        self.conv2 = nn.Conv2d(10, 40, kernel_size=5, stride=2, padding=2)
        self.conv3 = nn.Conv2d(40, 80, kernel_size=5, stride=1, padding=2)
        self.fc1 = nn.Linear(80 * 7 * 7, 50) # Adjusted based on input size
        and strides
        self.fc2 = nn.Linear(50, 10)

        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.5)
        self.softmax = nn.Softmax(dim=1)

    def forward(self, x):
        x = self.relu(self.conv1(x))
        x = self.relu(self.conv2(x))
        x = self.relu(self.conv3(x))
        x = x.view(x.size(0), -1) # Flatten
        x = self.relu(self.fc1(x))
        x = self.fc2(x)
        return x
```

```
[12]: # Early stopping implementation
class EarlyStopping:
    def __init__(self, patience=5, min_delta=0.0001, verbose=1):
        self.patience = patience
        self.min_delta = min_delta
        self.counter = 0
        self.best_accuracy = 0.0
        self.early_stop = False
        self.verbose = verbose
```



```

def __call__(self, current_accuracy):
    if current_accuracy - self.best_accuracy > self.min_delta:
        self.best_accuracy = current_accuracy
        self.counter = 0
    else:
        self.counter += 1
        if self.verbose:
            print(f'EarlyStopping counter: {self.counter} out of {self.
↪patience}')
        if self.counter >= self.patience:
            self.early_stop = True
    return self.early_stop

```

```

[13]: # Model Parameter Summarization
def summarize_layer_parameters(model):
    """
    Summarizes the number of parameters in each layer of a PyTorch model.
    """
    trainable_table = PrettyTable(["Module Name", "Parameters"])
    untrainable_table = PrettyTable(["Module Name", "Parameters"])
    total_params = {'train':0, 'untrain':0}
    for name, parameter in model.named_parameters():
        if parameter.requires_grad: # Only count trainable parameters
            params = parameter.numel()
            trainable_table.add_row([name, params])
            total_params['train'] += params
        else: # Only count untrainable parameters
            params = parameter.numel()
            untrainable_table.add_row([name, params])
            total_params['untrain'] += params

    print(trainable_table)
    print(f"\nTotal Trainable Parameters: {total_params['train']}")
    if total_params['untrain'] > 0:
        print(untrainable_table)
    print(f"\nTotal Un-Trainable Parameters: {total_params['untrain']}")

```

```

[14]: from torch.optim.lr_scheduler import ↪
↪ReduceLRonPlateau, CosineAnnealingLR, ExponentialLR, StepLR, MultiStepLR, LinearLR

# Model Evaluation
def evaluate_model(model, eval_loader, criterion=None):
    model.eval()
    eval_correct = 0
    eval_total = 0
    eval_loss = 0.0

```

```

with torch.no_grad():
    for data, target in eval_loader:
        data, target = data.to(device), target.to(device)
        output = model(data)
        _, predicted = torch.max(output.data, 1)
        eval_total += target.size(0)
        eval_correct += (predicted == target).sum().item()

        if criterion is not None:
            loss = criterion(output, target)
            eval_loss += loss.item()

eval_accuracy = 100 * eval_correct / eval_total
avg_eval_loss = eval_loss / len(eval_loader)

return eval_accuracy if criterion is None else (eval_accuracy,
↪avg_eval_loss)

# Model Training
def train_model(model, train_loader, valid_loader, criterion, optimizer,
↪epochs=100, patience=5, batch_scheduler=None, epoch_scheduler=None):
    history = {
        'train_loss': [], 'val_loss': [],
        'train_accuracy': [], 'val_accuracy': [],
        'learning_rate': []
    }
    early_stopping = EarlyStopping(patience=patience)
    assert batch_scheduler is None or epoch_scheduler is None

    # best_val_loss = float('inf')
    # patience_counter = 0

    for epoch in range(epochs):
        # Training phase
        model.train()
        train_loss = 0.0
        train_correct = 0
        train_total = 0

        for batch_idx, (data, target) in enumerate(train_loader):
            data, target = data.to(device), target.to(device)

            optimizer.zero_grad()
            output = model(data)

```

```

        # loss = criterion(output, torch.nn.functional.one_hot(target,
↪num_classes=10).float()) also works
        loss = criterion(output, target)

        loss.backward()
        optimizer.step()
        if batch_scheduler is not None:
            batch_scheduler.step()
            history['learning_rate'].append(optimizer.
↪state_dict()['param_groups'][0]['lr'])

        train_loss += loss.item()
        _, predicted = torch.max(output.data, 1)
        train_total += target.size(0)
        train_correct += (predicted == target).sum().item()

    train_accuracy = 100 * train_correct / train_total
    avg_train_loss = train_loss / len(train_loader)

    # Validation phase
    val_accuracy, avg_val_loss =
↪evaluate_model(model, valid_loader, criterion)

    # Store history
    history['train_loss'].append(avg_train_loss)
    history['val_loss'].append(avg_val_loss)
    history['train_accuracy'].append(train_accuracy / 100)
    history['val_accuracy'].append(val_accuracy / 100)

    train_msg = f' Train Loss: {avg_train_loss:.6f}, Train Acc:
↪{train_accuracy:.2f}%; '
    valid_msg = f' Val Loss: {avg_val_loss:.6f}, Val Acc: {val_accuracy:.
↪2f}%'

    print(f'Epoch {epoch+1}/{epochs}: ', train_msg, valid_msg)

    # Check early stopping
    if patience > 0:
        early_stopping(val_accuracy)
        if early_stopping.early_stop:
            print("Early stopping triggered")
            break
        else:
            torch.save(model.state_dict(), 'best_model.pth')
    else:
        torch.save(model.state_dict(), 'best_model.pth')

```

```

        if epoch_scheduler is not None:
            if isinstance(epoch_scheduler, ReduceLROnPlateau):
                epoch_scheduler.step(avg_val_loss)
            else:
                epoch_scheduler.step()
            history['learning_rate'].append(optimizer.
↪state_dict()['param_groups'][0]['lr'])

    # Load best model
    model.load_state_dict(torch.load('best_model.pth'))
    return history, model

```

```

[15]: # Initialize random seed
torch.manual_seed(4487)
random.seed(4487)
np.random.seed(4487)
if torch.cuda.is_available():
    torch.cuda.manual_seed(4487)
    torch.backends.cudnn.deterministic = True

# Create model instance
model = DeepCNN()

# Print model architecture
print(model)
summarize_layer_parameters(model)

# Move model to device (GPU if available)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
print(f"Using device: {device}")

# Prepare data - ensure correct shape for PyTorch (channel first)
# Assuming vtrainI, vtrainY, validI, validY are already defined as in previous
↪cells

# Convert from (batch, height, width, channels) to (batch, channels, height,
↪width)
if len(vtrainI.shape) == 4 and vtrainI.shape[-1] == 1: # If shape is (N, 28,
↪28, 1)
    vtrainI = vtrainI.transpose(0, 3, 1, 2)
    validI = validI.transpose(0, 3, 1, 2)

# Create DataLoaders
batch_size = 50
train_dataset = TensorDataset(vtrainI, vtrainY)

```

```

valid_dataset = TensorDataset(validI, validY)

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
valid_loader = DataLoader(valid_dataset, batch_size=batch_size, shuffle=False)

# Compile the network (equivalent in PyTorch)
criterion = nn.CrossEntropyLoss() # This includes softmax + cross entropy
optimizer = optim.SGD(model.parameters(), lr=0.02, momentum=0.9, nesterov=True)

# Train the model
history, model = train_model(model, train_loader, valid_loader, criterion,
    ↪optimizer, epochs=100)

# Plot the training history
plot_history(history)

```

```

DeepCNN(
  (conv1): Conv2d(1, 10, kernel_size=(5, 5), stride=(2, 2), padding=(2, 2))
  (conv2): Conv2d(10, 40, kernel_size=(5, 5), stride=(2, 2), padding=(2, 2))
  (conv3): Conv2d(40, 80, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
  (fc1): Linear(in_features=3920, out_features=50, bias=True)
  (fc2): Linear(in_features=50, out_features=10, bias=True)
  (relu): ReLU()
  (dropout): Dropout(p=0.5, inplace=False)
  (softmax): Softmax(dim=1)
)

```

Module Name	Parameters
conv1.weight	250
conv1.bias	10
conv2.weight	10000
conv2.bias	40
conv3.weight	80000
conv3.bias	80
fc1.weight	196000
fc1.bias	50
fc2.weight	500
fc2.bias	10

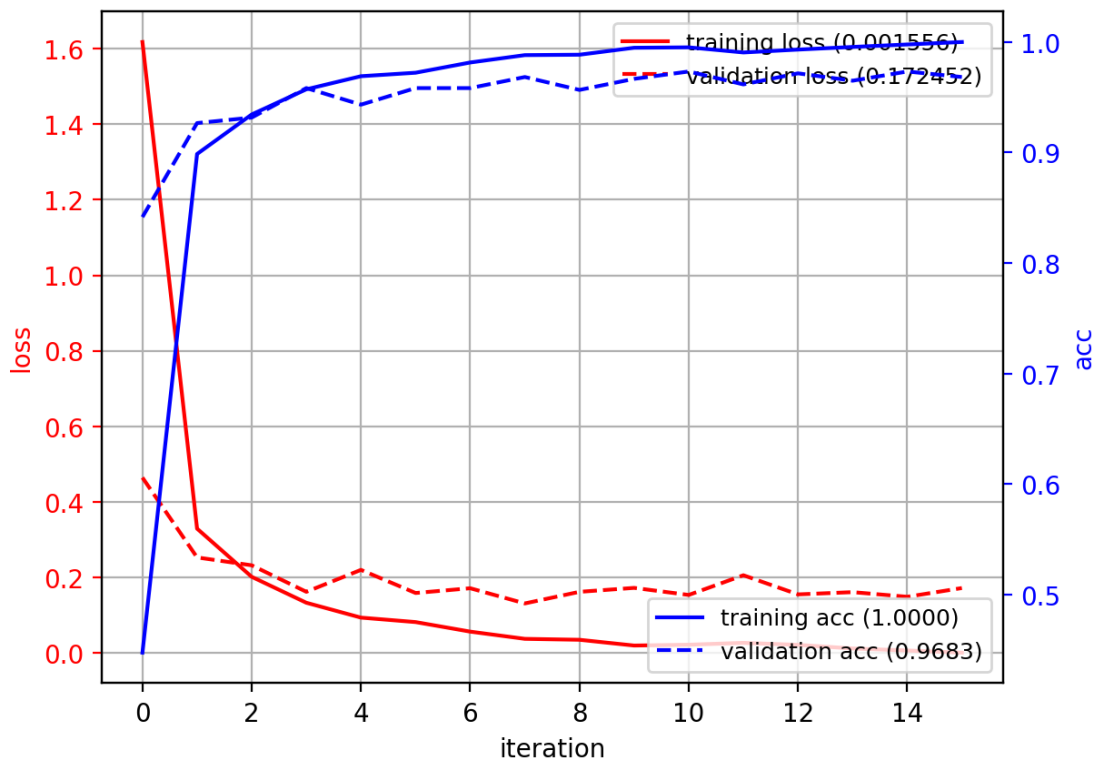
Total Trainable Parameters: 286940

Total Un-Trainable Parameters: 0

Using device: cuda

Epoch 1/100: Train Loss: 1.617102, Train Acc: 44.76%; Val Loss: 0.464670,
Val Acc: 84.17%

Epoch 2/100: Train Loss: 0.329649, Train Acc: 89.87%; Val Loss: 0.253477,
 Val Acc: 92.67%
 Epoch 3/100: Train Loss: 0.202198, Train Acc: 93.44%; Val Loss: 0.232633,
 Val Acc: 93.17%
 Epoch 4/100: Train Loss: 0.133558, Train Acc: 95.72%; Val Loss: 0.162527,
 Val Acc: 95.83%
 Epoch 5/100: Train Loss: 0.094230, Train Acc: 96.91%; Val Loss: 0.220510,
 Val Acc: 94.33%
 EarlyStopping counter: 1 out of 5
 Epoch 6/100: Train Loss: 0.082415, Train Acc: 97.22%; Val Loss: 0.159395,
 Val Acc: 95.83%
 EarlyStopping counter: 2 out of 5
 Epoch 7/100: Train Loss: 0.057166, Train Acc: 98.15%; Val Loss: 0.171946,
 Val Acc: 95.83%
 EarlyStopping counter: 3 out of 5
 Epoch 8/100: Train Loss: 0.038058, Train Acc: 98.81%; Val Loss: 0.131769,
 Val Acc: 96.83%
 Epoch 9/100: Train Loss: 0.035580, Train Acc: 98.85%; Val Loss: 0.162469,
 Val Acc: 95.67%
 EarlyStopping counter: 1 out of 5
 Epoch 10/100: Train Loss: 0.020369, Train Acc: 99.48%; Val Loss: 0.172894,
 Val Acc: 96.67%
 EarlyStopping counter: 2 out of 5
 Epoch 11/100: Train Loss: 0.022506, Train Acc: 99.52%; Val Loss: 0.154210,
 Val Acc: 97.33%
 Epoch 12/100: Train Loss: 0.027414, Train Acc: 99.06%; Val Loss: 0.206340,
 Val Acc: 96.17%
 EarlyStopping counter: 1 out of 5
 Epoch 13/100: Train Loss: 0.022148, Train Acc: 99.31%; Val Loss: 0.155686,
 Val Acc: 97.17%
 EarlyStopping counter: 2 out of 5
 Epoch 14/100: Train Loss: 0.012529, Train Acc: 99.56%; Val Loss: 0.161637,
 Val Acc: 96.50%
 EarlyStopping counter: 3 out of 5
 Epoch 15/100: Train Loss: 0.007216, Train Acc: 99.78%; Val Loss: 0.149385,
 Val Acc: 97.33%
 EarlyStopping counter: 4 out of 5
 Epoch 16/100: Train Loss: 0.001556, Train Acc: 100.00%; Val Loss: 0.172452,
 Val Acc: 96.83%
 EarlyStopping counter: 5 out of 5
 Early stopping triggered



```
[16]: test_dataset = TensorDataset(testI, torch.from_numpy(testY).long())
      test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)
      acc = evaluate_model(model, test_loader)

      print("test accuracy:", acc)
```

test accuracy: 96.7

11 Outline

- Going deeper
 - ReLU and Batchnorm
- **Optimization methods**
- Deep architectures and Image classification
- Transfer learning

12 Optimization with SGD

- Ideally, we would like to use all the samples to compute the gradient, but this is too time consuming.
- Use a *minibatch* (a few samples) at a time to estimate the gradient.
 - creates an unbiased estimator of the gradient.

- the variance (expected squared error) depends on the number of samples.
 - * i.e., the estimated gradient is *noisy*.

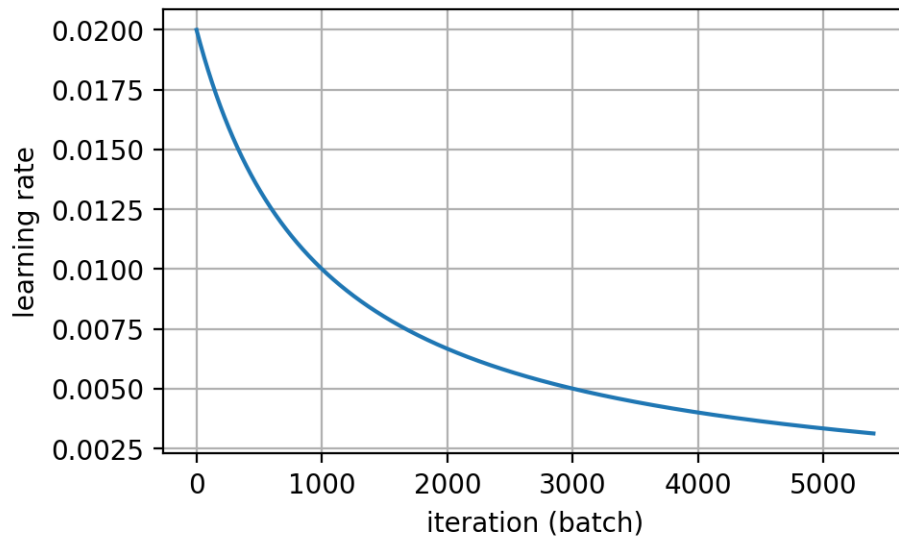
13 Learning rates

- **Problem:** For gradient descent, at the minimum we should have $\frac{dL}{d\mathbf{w}} = 0$.
- *What about for SGD?*
 - at the minimum $\frac{dL}{d\mathbf{w}} \neq 0$ because of noise in the gradient.
 - * SGD still moves around.
- **Solution:** reduce the learning rate during the epochs.
 - Examples: for iteration/epoch k ,
 - * **linear change:** $\eta_k = (1 - \alpha)\eta_0 + \alpha\eta_T$, where $\alpha = k/T$, and η_0, η_T given.
 - * **decay:** $\eta_k = \frac{1}{1+\delta k}\eta_0$, where $0 < \delta < 1$.
 - we want a small learning rate when we are close to the minimum.
 - * needs to be set empirically by examining the learning curves.

14 Example: Keras decay

- use the built-in `decay` parameter.
 - applied after each batch.

```
[17]: plt.figure(figsize=(5,3))
      its = arange(0,50*5400/50) # 50 epochs, 5400/50 iterations per epoch
      lr = 0.02*(1./(1+its*1e-3))
      plt.plot(its, lr)
      plt.grid(); plt.xlabel('iteration (batch)'); plt.ylabel('learning rate');
      plt.show()
```




```
[18]: # Create model instance
model = DeepCNN()

# Move model to device (GPU if available)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
print(f"Using device: {device}")

criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=0.02, momentum=0.9, nesterov=True)
scheduler = ExponentialLR(optimizer, gamma=math.pow(0.95, 1/108))

# Train the model
history, model = train_model(model, train_loader, valid_loader, criterion,
    ↪optimizer, epochs=50, patience=-1, batch_scheduler=scheduler)
```

Using device: cuda

Epoch 1/50:	Train Loss: 1.716768, Train Acc: 40.04%;	Val Loss: 0.419060, Val Acc: 88.17%
Epoch 2/50:	Train Loss: 0.325775, Train Acc: 89.72%;	Val Loss: 0.260520, Val Acc: 93.17%
Epoch 3/50:	Train Loss: 0.202823, Train Acc: 93.93%;	Val Loss: 0.179392, Val Acc: 93.50%
Epoch 4/50:	Train Loss: 0.137527, Train Acc: 95.85%;	Val Loss: 0.182461, Val Acc: 95.00%
Epoch 5/50:	Train Loss: 0.091641, Train Acc: 97.06%;	Val Loss: 0.180758, Val Acc: 95.50%
Epoch 6/50:	Train Loss: 0.069151, Train Acc: 97.81%;	Val Loss: 0.158545, Val Acc: 95.00%
Epoch 7/50:	Train Loss: 0.051072, Train Acc: 98.31%;	Val Loss: 0.152725, Val Acc: 96.17%
Epoch 8/50:	Train Loss: 0.030280, Train Acc: 99.13%;	Val Loss: 0.161747, Val Acc: 96.00%
Epoch 9/50:	Train Loss: 0.022639, Train Acc: 99.33%;	Val Loss: 0.173594, Val Acc: 96.33%
Epoch 10/50:	Train Loss: 0.015947, Train Acc: 99.56%;	Val Loss: 0.162648, Val Acc: 96.67%
Epoch 11/50:	Train Loss: 0.011119, Train Acc: 99.63%;	Val Loss: 0.170903, Val Acc: 96.67%
Epoch 12/50:	Train Loss: 0.008652, Train Acc: 99.74%;	Val Loss: 0.166767, Val Acc: 96.67%
Epoch 13/50:	Train Loss: 0.002905, Train Acc: 99.98%;	Val Loss: 0.181154, Val Acc: 96.67%
Epoch 14/50:	Train Loss: 0.001629, Train Acc: 99.98%;	Val Loss: 0.172741, Val Acc: 97.33%
Epoch 15/50:	Train Loss: 0.001207, Train Acc: 100.00%;	Val Loss: 0.186913, Val Acc: 96.83%

Epoch 16/50: Train Loss: 0.000863, Train Acc: 100.00%; Val Loss: 0.189911, Val Acc: 96.83%

Epoch 17/50: Train Loss: 0.000713, Train Acc: 100.00%; Val Loss: 0.192111, Val Acc: 97.00%

Epoch 18/50: Train Loss: 0.000625, Train Acc: 100.00%; Val Loss: 0.194995, Val Acc: 97.00%

Epoch 19/50: Train Loss: 0.000567, Train Acc: 100.00%; Val Loss: 0.199100, Val Acc: 96.83%

Epoch 20/50: Train Loss: 0.000514, Train Acc: 100.00%; Val Loss: 0.199754, Val Acc: 97.17%

Epoch 21/50: Train Loss: 0.000472, Train Acc: 100.00%; Val Loss: 0.202644, Val Acc: 97.00%

Epoch 22/50: Train Loss: 0.000440, Train Acc: 100.00%; Val Loss: 0.205207, Val Acc: 96.83%

Epoch 23/50: Train Loss: 0.000418, Train Acc: 100.00%; Val Loss: 0.205804, Val Acc: 96.83%

Epoch 24/50: Train Loss: 0.000397, Train Acc: 100.00%; Val Loss: 0.206984, Val Acc: 97.33%

Epoch 25/50: Train Loss: 0.000374, Train Acc: 100.00%; Val Loss: 0.209511, Val Acc: 97.00%

Epoch 26/50: Train Loss: 0.000354, Train Acc: 100.00%; Val Loss: 0.210647, Val Acc: 97.17%

Epoch 27/50: Train Loss: 0.000342, Train Acc: 100.00%; Val Loss: 0.211823, Val Acc: 97.00%

Epoch 28/50: Train Loss: 0.000330, Train Acc: 100.00%; Val Loss: 0.213149, Val Acc: 97.17%

Epoch 29/50: Train Loss: 0.000316, Train Acc: 100.00%; Val Loss: 0.214542, Val Acc: 97.17%

Epoch 30/50: Train Loss: 0.000307, Train Acc: 100.00%; Val Loss: 0.215214, Val Acc: 97.17%

Epoch 31/50: Train Loss: 0.000298, Train Acc: 100.00%; Val Loss: 0.215653, Val Acc: 97.17%

Epoch 32/50: Train Loss: 0.000289, Train Acc: 100.00%; Val Loss: 0.216484, Val Acc: 97.17%

Epoch 33/50: Train Loss: 0.000281, Train Acc: 100.00%; Val Loss: 0.217421, Val Acc: 97.17%

Epoch 34/50: Train Loss: 0.000275, Train Acc: 100.00%; Val Loss: 0.218286, Val Acc: 97.17%

Epoch 35/50: Train Loss: 0.000270, Train Acc: 100.00%; Val Loss: 0.218745, Val Acc: 97.17%

Epoch 36/50: Train Loss: 0.000264, Train Acc: 100.00%; Val Loss: 0.219660, Val Acc: 97.17%

Epoch 37/50: Train Loss: 0.000259, Train Acc: 100.00%; Val Loss: 0.220107, Val Acc: 97.00%

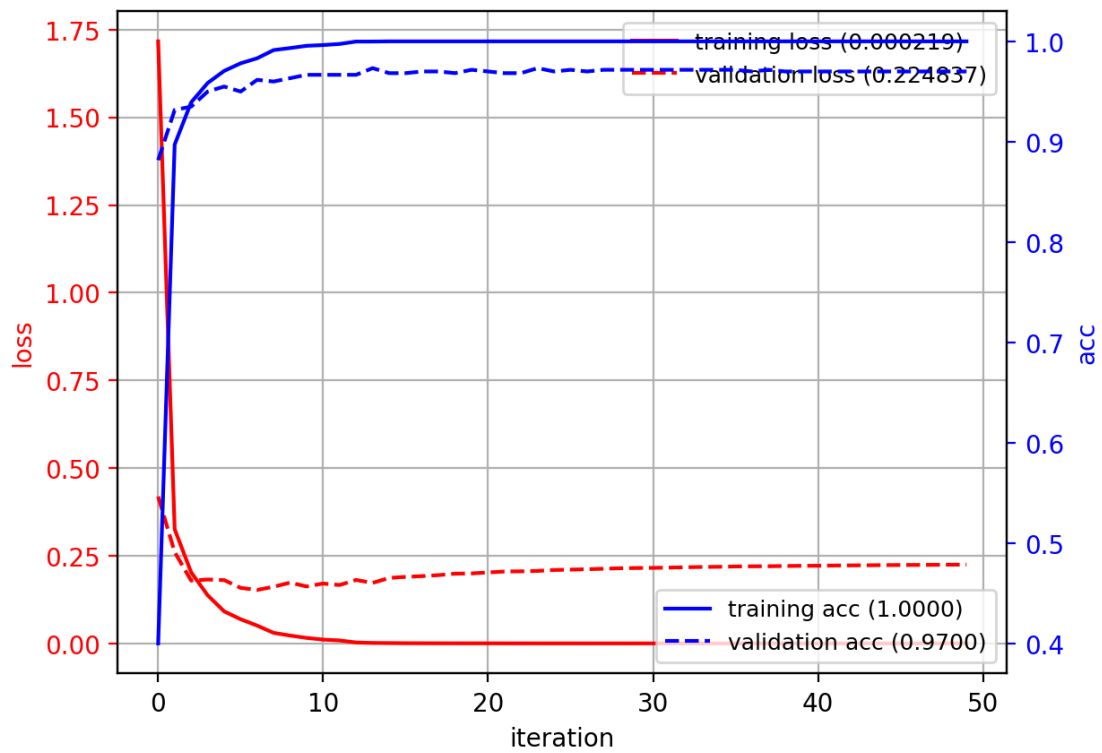
Epoch 38/50: Train Loss: 0.000254, Train Acc: 100.00%; Val Loss: 0.220290, Val Acc: 97.17%

Epoch 39/50: Train Loss: 0.000250, Train Acc: 100.00%; Val Loss: 0.220998, Val Acc: 97.00%

Epoch 40/50: Train Loss: 0.000246, Train Acc: 100.00%; Val Loss: 0.221544, Val Acc: 97.00%
Epoch 41/50: Train Loss: 0.000243, Train Acc: 100.00%; Val Loss: 0.221706, Val Acc: 97.00%
Epoch 42/50: Train Loss: 0.000239, Train Acc: 100.00%; Val Loss: 0.222382, Val Acc: 97.00%
Epoch 43/50: Train Loss: 0.000235, Train Acc: 100.00%; Val Loss: 0.222725, Val Acc: 97.00%
Epoch 44/50: Train Loss: 0.000232, Train Acc: 100.00%; Val Loss: 0.223023, Val Acc: 97.00%
Epoch 45/50: Train Loss: 0.000230, Train Acc: 100.00%; Val Loss: 0.223515, Val Acc: 97.00%
Epoch 46/50: Train Loss: 0.000228, Train Acc: 100.00%; Val Loss: 0.223785, Val Acc: 97.00%
Epoch 47/50: Train Loss: 0.000225, Train Acc: 100.00%; Val Loss: 0.224014, Val Acc: 97.00%
Epoch 48/50: Train Loss: 0.000223, Train Acc: 100.00%; Val Loss: 0.224324, Val Acc: 97.00%
Epoch 49/50: Train Loss: 0.000221, Train Acc: 100.00%; Val Loss: 0.224666, Val Acc: 97.00%
Epoch 50/50: Train Loss: 0.000219, Train Acc: 100.00%; Val Loss: 0.224837, Val Acc: 97.00%

```
[19]: # Plot the training history
      plot_history(history)

      acc = evaluate_model(model, test_loader)
      print("test accuracy:", acc)
```



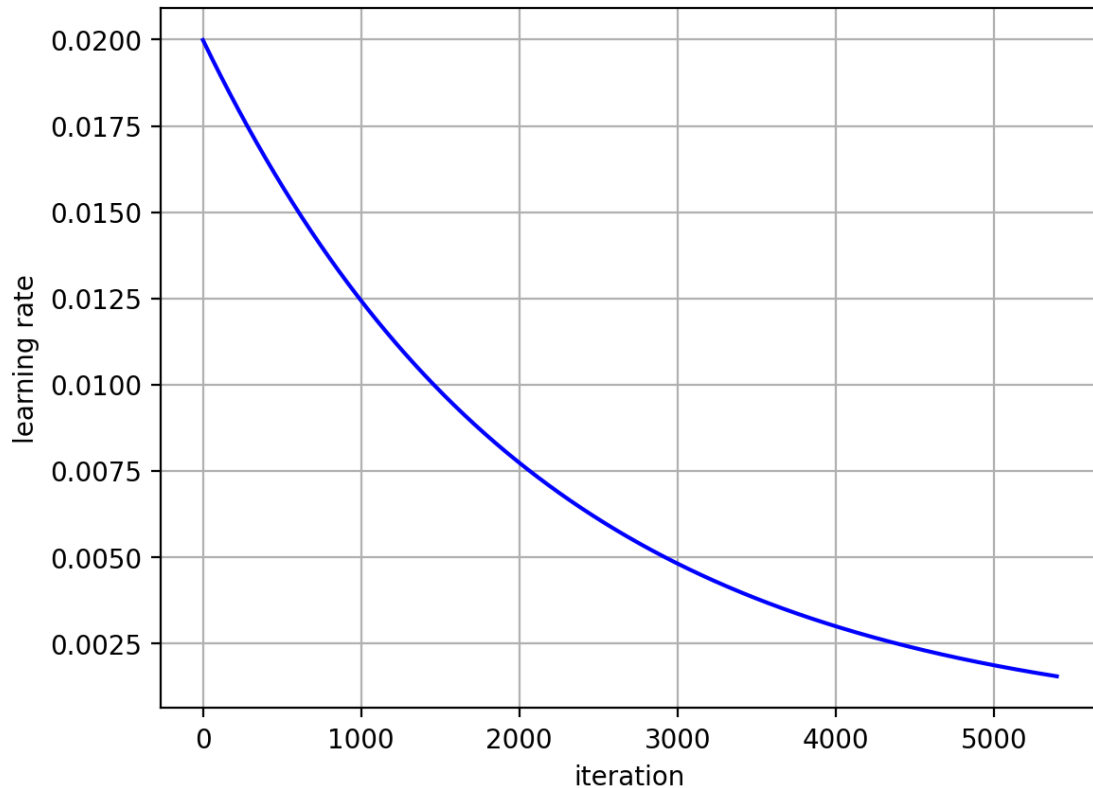
test accuracy: 96.62

```
[20]: def plot_lr(history):
    fig, ax1 = plt.subplots()

    ax1.plot(history['learning_rate'], 'b')
    ax1.grid(True)
    ax1.set_xlabel('iteration')
    ax1.set_ylabel('learning rate')

    plt.show()

plot_lr(history)
```



15 Adaptive schedule

- reduce the learning rate when the validation loss no longer improves
 - similar to early stopping criteria
- implemented as a callback function

```
[21]: # Create model instance
model = DeepCNN()

# Move model to device (GPU if available)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
print(f"Using device: {device}")

criterion = nn.CrossEntropyLoss() # This includes softmax + cross entropy
optimizer = optim.SGD(model.parameters(), lr=0.02, momentum=0.9, nesterov=True)
# reduce LR by a factor of 0.1, if no change in 5 epochs
scheduler = ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=5)

# Train the model
```

```
history, model = train_model(model, train_loader, valid_loader, criterion, □
                               ↪optimizer, epochs=50, patience=-1, epoch_scheduler=scheduler)
```

Using device: cuda

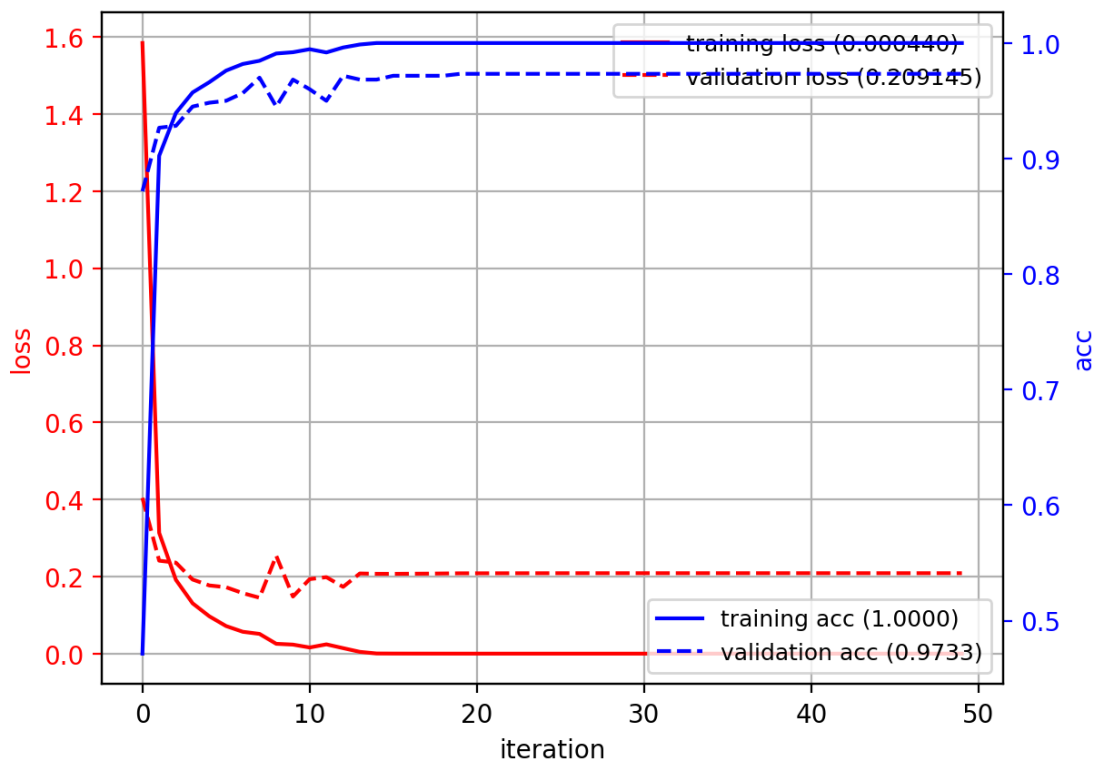
Epoch 1/50:	Train Loss: 1.584269, Train Acc: 47.15%;	Val Loss: 0.404391, Val Acc: 87.17%
Epoch 2/50:	Train Loss: 0.313745, Train Acc: 90.24%;	Val Loss: 0.241543, Val Acc: 92.67%
Epoch 3/50:	Train Loss: 0.191844, Train Acc: 93.94%;	Val Loss: 0.236455, Val Acc: 92.83%
Epoch 4/50:	Train Loss: 0.130987, Train Acc: 95.72%;	Val Loss: 0.192679, Val Acc: 94.50%
Epoch 5/50:	Train Loss: 0.097266, Train Acc: 96.61%;	Val Loss: 0.177332, Val Acc: 94.83%
Epoch 6/50:	Train Loss: 0.072049, Train Acc: 97.61%;	Val Loss: 0.172696, Val Acc: 95.00%
Epoch 7/50:	Train Loss: 0.057187, Train Acc: 98.19%;	Val Loss: 0.157365, Val Acc: 95.67%
Epoch 8/50:	Train Loss: 0.051671, Train Acc: 98.46%;	Val Loss: 0.145465, Val Acc: 97.00%
Epoch 9/50:	Train Loss: 0.025971, Train Acc: 99.09%;	Val Loss: 0.254315, Val Acc: 94.50%
Epoch 10/50:	Train Loss: 0.023875, Train Acc: 99.20%;	Val Loss: 0.148523, Val Acc: 96.83%
Epoch 11/50:	Train Loss: 0.016409, Train Acc: 99.46%;	Val Loss: 0.193763, Val Acc: 96.00%
Epoch 12/50:	Train Loss: 0.024521, Train Acc: 99.19%;	Val Loss: 0.198806, Val Acc: 95.00%
Epoch 13/50:	Train Loss: 0.014728, Train Acc: 99.61%;	Val Loss: 0.173626, Val Acc: 97.17%
Epoch 14/50:	Train Loss: 0.005073, Train Acc: 99.87%;	Val Loss: 0.208359, Val Acc: 96.83%
Epoch 15/50:	Train Loss: 0.000861, Train Acc: 100.00%;	Val Loss: 0.207530, Val Acc: 96.83%
Epoch 16/50:	Train Loss: 0.000659, Train Acc: 100.00%;	Val Loss: 0.207451, Val Acc: 97.17%
Epoch 17/50:	Train Loss: 0.000588, Train Acc: 100.00%;	Val Loss: 0.207748, Val Acc: 97.17%
Epoch 18/50:	Train Loss: 0.000541, Train Acc: 100.00%;	Val Loss: 0.207970, Val Acc: 97.17%
Epoch 19/50:	Train Loss: 0.000506, Train Acc: 100.00%;	Val Loss: 0.208384, Val Acc: 97.17%
Epoch 20/50:	Train Loss: 0.000477, Train Acc: 100.00%;	Val Loss: 0.208821, Val Acc: 97.33%
Epoch 21/50:	Train Loss: 0.000454, Train Acc: 100.00%;	Val Loss: 0.208852, Val Acc: 97.33%
Epoch 22/50:	Train Loss: 0.000452, Train Acc: 100.00%;	Val Loss: 0.208900, Val Acc: 97.33%

Val Acc: 97.33%
 Epoch 23/50: Train Loss: 0.000449, Train Acc: 100.00%; Val Loss: 0.208964,
 Val Acc: 97.33%
 Epoch 24/50: Train Loss: 0.000447, Train Acc: 100.00%; Val Loss: 0.209015,
 Val Acc: 97.33%
 Epoch 25/50: Train Loss: 0.000445, Train Acc: 100.00%; Val Loss: 0.209046,
 Val Acc: 97.33%
 Epoch 26/50: Train Loss: 0.000443, Train Acc: 100.00%; Val Loss: 0.209096,
 Val Acc: 97.33%
 Epoch 27/50: Train Loss: 0.000441, Train Acc: 100.00%; Val Loss: 0.209106,
 Val Acc: 97.33%
 Epoch 28/50: Train Loss: 0.000441, Train Acc: 100.00%; Val Loss: 0.209115,
 Val Acc: 97.33%
 Epoch 29/50: Train Loss: 0.000441, Train Acc: 100.00%; Val Loss: 0.209125,
 Val Acc: 97.33%
 Epoch 30/50: Train Loss: 0.000441, Train Acc: 100.00%; Val Loss: 0.209133,
 Val Acc: 97.33%
 Epoch 31/50: Train Loss: 0.000440, Train Acc: 100.00%; Val Loss: 0.209137,
 Val Acc: 97.33%
 Epoch 32/50: Train Loss: 0.000440, Train Acc: 100.00%; Val Loss: 0.209143,
 Val Acc: 97.33%
 Epoch 33/50: Train Loss: 0.000440, Train Acc: 100.00%; Val Loss: 0.209143,
 Val Acc: 97.33%
 Epoch 34/50: Train Loss: 0.000440, Train Acc: 100.00%; Val Loss: 0.209143,
 Val Acc: 97.33%
 Epoch 35/50: Train Loss: 0.000440, Train Acc: 100.00%; Val Loss: 0.209143,
 Val Acc: 97.33%
 Epoch 36/50: Train Loss: 0.000440, Train Acc: 100.00%; Val Loss: 0.209144,
 Val Acc: 97.33%
 Epoch 37/50: Train Loss: 0.000440, Train Acc: 100.00%; Val Loss: 0.209145,
 Val Acc: 97.33%
 Epoch 38/50: Train Loss: 0.000440, Train Acc: 100.00%; Val Loss: 0.209145,
 Val Acc: 97.33%
 Epoch 39/50: Train Loss: 0.000440, Train Acc: 100.00%; Val Loss: 0.209145,
 Val Acc: 97.33%
 Epoch 40/50: Train Loss: 0.000440, Train Acc: 100.00%; Val Loss: 0.209145,
 Val Acc: 97.33%
 Epoch 41/50: Train Loss: 0.000440, Train Acc: 100.00%; Val Loss: 0.209145,
 Val Acc: 97.33%
 Epoch 42/50: Train Loss: 0.000440, Train Acc: 100.00%; Val Loss: 0.209145,
 Val Acc: 97.33%
 Epoch 43/50: Train Loss: 0.000440, Train Acc: 100.00%; Val Loss: 0.209145,
 Val Acc: 97.33%
 Epoch 44/50: Train Loss: 0.000440, Train Acc: 100.00%; Val Loss: 0.209145,
 Val Acc: 97.33%
 Epoch 45/50: Train Loss: 0.000440, Train Acc: 100.00%; Val Loss: 0.209145,
 Val Acc: 97.33%
 Epoch 46/50: Train Loss: 0.000440, Train Acc: 100.00%; Val Loss: 0.209145,

Val Acc: 97.33%
 Epoch 47/50: Train Loss: 0.000440, Train Acc: 100.00%; Val Loss: 0.209145, Val Acc: 97.33%
 Epoch 48/50: Train Loss: 0.000440, Train Acc: 100.00%; Val Loss: 0.209145, Val Acc: 97.33%
 Epoch 49/50: Train Loss: 0.000440, Train Acc: 100.00%; Val Loss: 0.209145, Val Acc: 97.33%
 Epoch 50/50: Train Loss: 0.000440, Train Acc: 100.00%; Val Loss: 0.209145, Val Acc: 97.33%

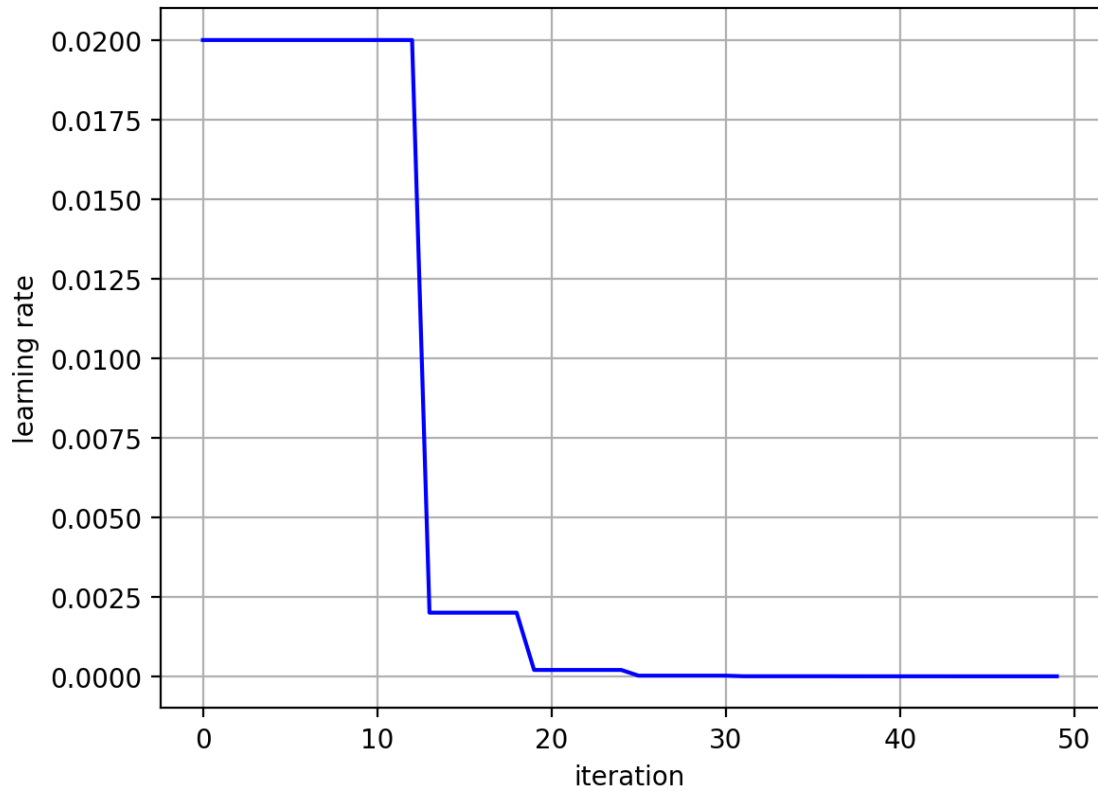
```
[22]: # Plot the training history
plot_history(history)

acc = evaluate_model(model, test_loader)
print("test accuracy:", acc)
```



test accuracy: 96.42

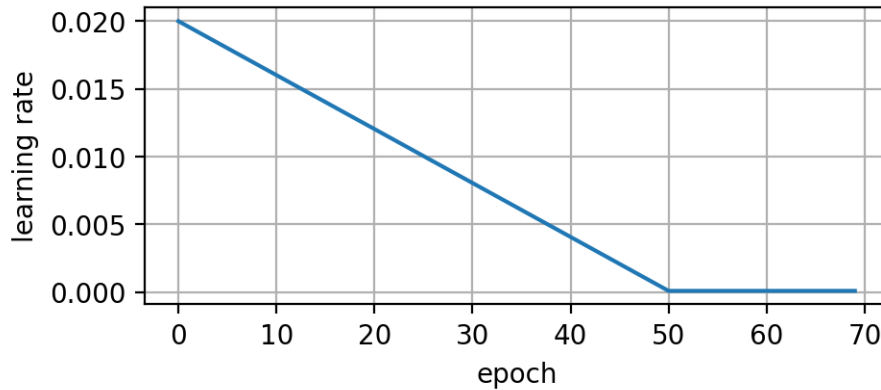
```
[23]: plot_lr(history)
```

16 Fixed schedule

- specify our own schedule using callback `LearningRateScheduler`
- pass a schedule function
- inputs are the epoch and current learning rate.
- outputs the learning rate for this epoch.

```
[24]: def sc(epoch, curlr):
        alpha = minimum(epoch/50, 1.)
        return 0.02*(1-alpha)+0.0001*alpha
    epoch = arange(0,70)
    plt.figure(figsize=(5,2))
    plt.plot(epoch, sc(epoch,None))
    plt.grid(); plt.xlabel('epoch'); plt.ylabel('learning rate');plt.show()
```



```
[25]: # Create model instance
model = DeepCNN()

# Move model to device (GPU if available)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
print(f"Using device: {device}")

criterion = nn.CrossEntropyLoss() # This includes softmax + cross entropy
optimizer = optim.SGD(model.parameters(), lr=0.02, momentum=0.9, nesterov=True)
# reduce LR by a factor of 0.1, if no change in 5 epochs
scheduler = LinearLR(optimizer, start_factor=1.0, end_factor=0.05,
    ↪total_iters=50)

# Train the model
history, model = train_model(model, train_loader, valid_loader, criterion,
    ↪optimizer, epochs=50, patience=-1, epoch_scheduler=scheduler)
```

Using device: cuda

Epoch 1/50:	Train Loss: 1.561114, Train Acc: 45.91%;	Val Loss: 0.400893, Val Acc: 87.00%
Epoch 2/50:	Train Loss: 0.305708, Train Acc: 90.65%;	Val Loss: 0.233811, Val Acc: 92.83%
Epoch 3/50:	Train Loss: 0.183516, Train Acc: 94.56%;	Val Loss: 0.199219, Val Acc: 94.33%
Epoch 4/50:	Train Loss: 0.128340, Train Acc: 95.87%;	Val Loss: 0.186400, Val Acc: 94.83%
Epoch 5/50:	Train Loss: 0.085602, Train Acc: 97.31%;	Val Loss: 0.175606, Val Acc: 96.00%
Epoch 6/50:	Train Loss: 0.068440, Train Acc: 97.72%;	Val Loss: 0.208534, Val Acc: 95.17%
Epoch 7/50:	Train Loss: 0.053286, Train Acc: 98.31%;	Val Loss: 0.150136, Val Acc: 97.17%

Epoch 8/50: Train Loss: 0.033672, Train Acc: 99.04%; Val Loss: 0.166403, Val Acc: 95.33%

Epoch 9/50: Train Loss: 0.034847, Train Acc: 98.89%; Val Loss: 0.157852, Val Acc: 96.17%

Epoch 10/50: Train Loss: 0.019101, Train Acc: 99.43%; Val Loss: 0.181433, Val Acc: 96.67%

Epoch 11/50: Train Loss: 0.021414, Train Acc: 99.28%; Val Loss: 0.172300, Val Acc: 97.33%

Epoch 12/50: Train Loss: 0.006556, Train Acc: 99.85%; Val Loss: 0.205704, Val Acc: 96.83%

Epoch 13/50: Train Loss: 0.006376, Train Acc: 99.80%; Val Loss: 0.172653, Val Acc: 97.67%

Epoch 14/50: Train Loss: 0.001764, Train Acc: 99.96%; Val Loss: 0.188386, Val Acc: 97.33%

Epoch 15/50: Train Loss: 0.000815, Train Acc: 100.00%; Val Loss: 0.193944, Val Acc: 97.33%

Epoch 16/50: Train Loss: 0.000452, Train Acc: 100.00%; Val Loss: 0.198022, Val Acc: 97.33%

Epoch 17/50: Train Loss: 0.000335, Train Acc: 100.00%; Val Loss: 0.201504, Val Acc: 97.67%

Epoch 18/50: Train Loss: 0.000283, Train Acc: 100.00%; Val Loss: 0.206944, Val Acc: 97.50%

Epoch 19/50: Train Loss: 0.000247, Train Acc: 100.00%; Val Loss: 0.210983, Val Acc: 97.50%

Epoch 20/50: Train Loss: 0.000224, Train Acc: 100.00%; Val Loss: 0.212869, Val Acc: 97.50%

Epoch 21/50: Train Loss: 0.000201, Train Acc: 100.00%; Val Loss: 0.214884, Val Acc: 97.50%

Epoch 22/50: Train Loss: 0.000186, Train Acc: 100.00%; Val Loss: 0.218093, Val Acc: 97.50%

Epoch 23/50: Train Loss: 0.000173, Train Acc: 100.00%; Val Loss: 0.219498, Val Acc: 97.67%

Epoch 24/50: Train Loss: 0.000162, Train Acc: 100.00%; Val Loss: 0.220969, Val Acc: 97.67%

Epoch 25/50: Train Loss: 0.000153, Train Acc: 100.00%; Val Loss: 0.222589, Val Acc: 97.67%

Epoch 26/50: Train Loss: 0.000143, Train Acc: 100.00%; Val Loss: 0.224058, Val Acc: 97.67%

Epoch 27/50: Train Loss: 0.000137, Train Acc: 100.00%; Val Loss: 0.225604, Val Acc: 97.67%

Epoch 28/50: Train Loss: 0.000130, Train Acc: 100.00%; Val Loss: 0.226975, Val Acc: 97.67%

Epoch 29/50: Train Loss: 0.000125, Train Acc: 100.00%; Val Loss: 0.227996, Val Acc: 97.67%

Epoch 30/50: Train Loss: 0.000121, Train Acc: 100.00%; Val Loss: 0.228760, Val Acc: 97.67%

Epoch 31/50: Train Loss: 0.000116, Train Acc: 100.00%; Val Loss: 0.229694, Val Acc: 97.67%

```

Epoch 32/50:   Train Loss: 0.000112, Train Acc: 100.00%;   Val Loss: 0.230692,
Val Acc: 97.67%
Epoch 33/50:   Train Loss: 0.000109, Train Acc: 100.00%;   Val Loss: 0.231587,
Val Acc: 97.67%
Epoch 34/50:   Train Loss: 0.000106, Train Acc: 100.00%;   Val Loss: 0.232389,
Val Acc: 97.67%
Epoch 35/50:   Train Loss: 0.000103, Train Acc: 100.00%;   Val Loss: 0.233086,
Val Acc: 97.67%
Epoch 36/50:   Train Loss: 0.000100, Train Acc: 100.00%;   Val Loss: 0.233730,
Val Acc: 97.67%
Epoch 37/50:   Train Loss: 0.000098, Train Acc: 100.00%;   Val Loss: 0.234420,
Val Acc: 97.67%
Epoch 38/50:   Train Loss: 0.000096, Train Acc: 100.00%;   Val Loss: 0.234842,
Val Acc: 97.67%
Epoch 39/50:   Train Loss: 0.000094, Train Acc: 100.00%;   Val Loss: 0.235350,
Val Acc: 97.67%
Epoch 40/50:   Train Loss: 0.000092, Train Acc: 100.00%;   Val Loss: 0.235751,
Val Acc: 97.67%
Epoch 41/50:   Train Loss: 0.000091, Train Acc: 100.00%;   Val Loss: 0.236222,
Val Acc: 97.67%
Epoch 42/50:   Train Loss: 0.000090, Train Acc: 100.00%;   Val Loss: 0.236660,
Val Acc: 97.67%
Epoch 43/50:   Train Loss: 0.000088, Train Acc: 100.00%;   Val Loss: 0.236924,
Val Acc: 97.67%
Epoch 44/50:   Train Loss: 0.000087, Train Acc: 100.00%;   Val Loss: 0.237250,
Val Acc: 97.67%
Epoch 45/50:   Train Loss: 0.000086, Train Acc: 100.00%;   Val Loss: 0.237504,
Val Acc: 97.67%
Epoch 46/50:   Train Loss: 0.000086, Train Acc: 100.00%;   Val Loss: 0.237803,
Val Acc: 97.67%
Epoch 47/50:   Train Loss: 0.000085, Train Acc: 100.00%;   Val Loss: 0.237978,
Val Acc: 97.67%
Epoch 48/50:   Train Loss: 0.000084, Train Acc: 100.00%;   Val Loss: 0.238225,
Val Acc: 97.67%
Epoch 49/50:   Train Loss: 0.000084, Train Acc: 100.00%;   Val Loss: 0.238314,
Val Acc: 97.67%
Epoch 50/50:   Train Loss: 0.000083, Train Acc: 100.00%;   Val Loss: 0.238428,
Val Acc: 97.67%

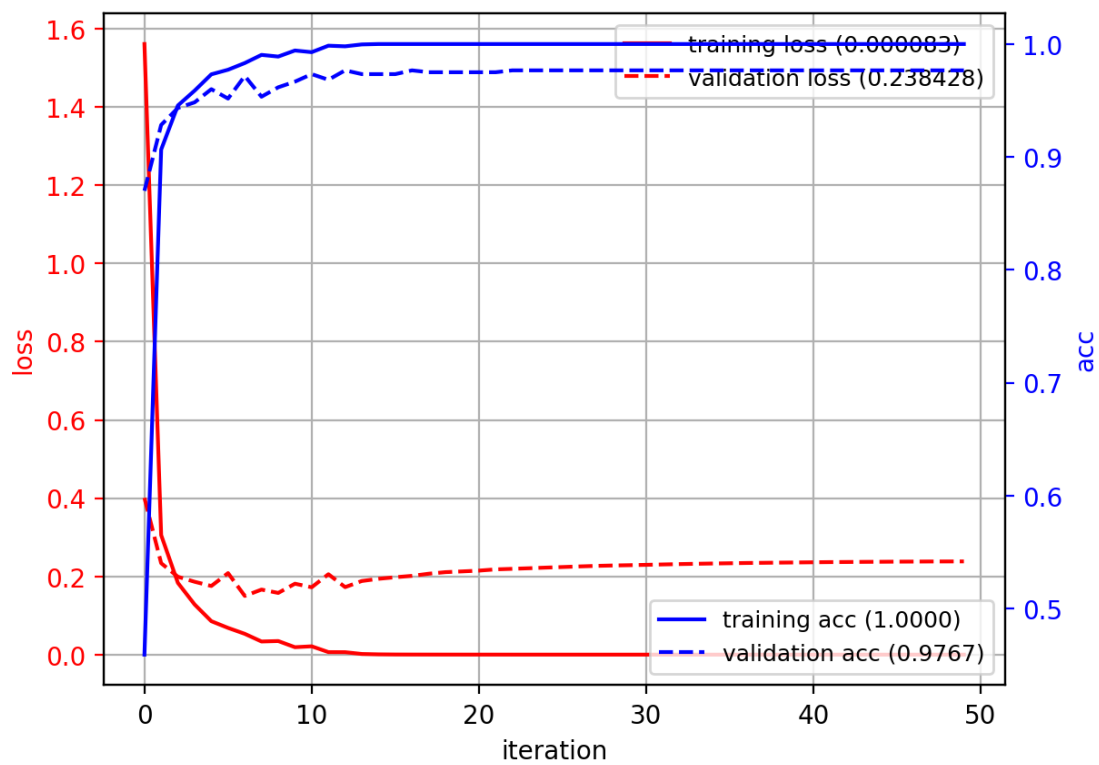
```

```

[26]: # Plot the training history
      plot_history(history)

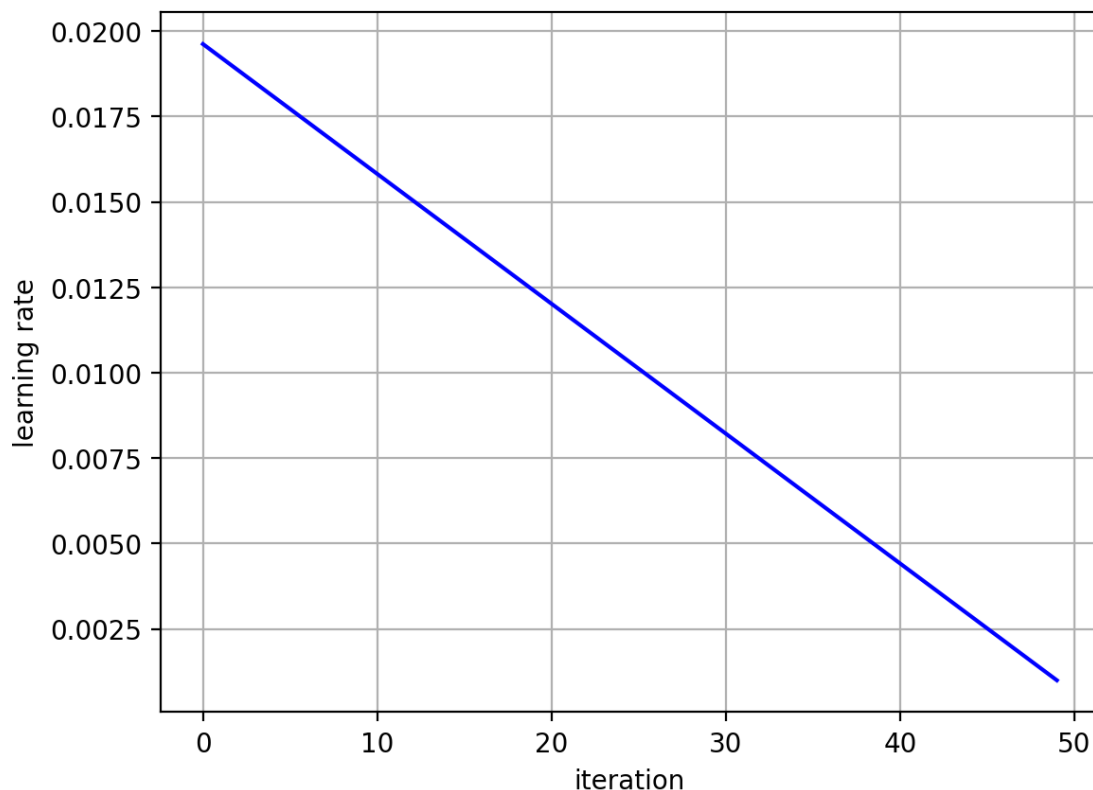
      acc = evaluate_model(model, test_loader)
      print("test accuracy:", acc)

```



test accuracy: 96.46

```
[27]: plot_lr(history)
```



17 Momentum

- **Problem:** The estimated gradient is noisy, can jump around.
- **Solution:** keep a running average of the gradients across mini-batches.
 - velocity: $\mathbf{v}^{(t)} = \alpha \mathbf{v}^{(t-1)} - \eta \left. \frac{dL}{d\mathbf{w}} \right|_{\mathbf{w}^{(t-1)}}$
 - * accumulate the gradients
 - * α is the momentum hyperparameter; how much it exponentially decays.
 - parameter update: $\mathbf{w}^{(t)} \leftarrow \mathbf{w}^{(t-1)} + \mathbf{v}^{(t)}$
- Example:
 - red path is using momentum
 - black arrows show the gradient directions at each step
 - without momentum, the path would oscillate wildly.

18 Nesterov Momentum

- Compute the gradient *after* the current velocity is applied.
 - interim update: $\tilde{\mathbf{w}} = \mathbf{w}^{(t-1)} + \alpha \mathbf{v}^{(t-1)}$
 - velocity: $\mathbf{v}^{(t)} = \alpha \mathbf{v}^{(t-1)} - \eta \left. \frac{dL}{d\mathbf{w}} \right|_{\tilde{\mathbf{w}}}$
 - parameter update: $\mathbf{w}^{(t)} \leftarrow \mathbf{w}^{(t-1)} + \mathbf{v}^{(t)}$
- Adds a *correction factor* to improve convergence (for convex batch case)

19 Why does SGD work?

- The loss function has many local minima
- SGD adds “noise” to the true gradient.
 - the noise allows escaping/avoiding/jumping over small local minima.
- Example:
 - red arrow = true gradient
 - green arrow = added noise
 - black arrow = computed gradient

20 SGD smoothes the loss function

- the added gradient noise is equivalent to convolving the loss function with the noise density.
- higher learning rate $\rightarrow$ larger noise $\rightarrow$ smoother loss
- smoother loss removes the local minimum, making it easier to get near the global minimum.
 - but not exactly on it.
- need to reduce the learning rate in stages to converge to the global optimum.

21 Optimization with Adaptive Learning Rates

- Introduce a separate learning rate for each parameter, and automatically adapt the learning rates during optimization.
- **AdaGrad** (`torch.optim.Adagrad`)
 - adapt individual learning rates by dividing by the square-root of the gradient energy accumulated over the iterations.
 - * $\mathbf{g} = \frac{dL}{d\mathbf{w}}$
 - * $\mathbf{r}^{(t)} = \mathbf{r}^{(t-1)} + \mathbf{g}^2$
 - * $\mathbf{w}^{(t)} = \mathbf{w}^{(t-1)} - \frac{\eta}{\delta + \sqrt{\mathbf{r}}} \mathbf{g}$
 - (operations are element-wise)
- **RMSProp** (`torch.optim.RMSprop`)
 - use exponential decay on the accumulated energy:
 - * $\mathbf{r}^{(t)} = \rho \mathbf{r}^{(t-1)} + (1 - \rho) \mathbf{g}^2$
- **Adam** (`torch.optim.Adam`)
 - use momentum with exponential weighting to estimate the gradient and gradient energy.
 - * $\mathbf{s}^{(t)} = \rho_1 \mathbf{s}^{(t-1)} + (1 - \rho_1) \mathbf{g}$
 - * $\mathbf{r}^{(t)} = \rho_2 \mathbf{r}^{(t-1)} + (1 - \rho_2) \mathbf{g}^2$
 - adds a bias correction for these two estimates.
 - * $\hat{\mathbf{s}}^{(t)} = \frac{1}{1 - \rho_1^t} \mathbf{s}^{(t)}$
 - * $\hat{\mathbf{r}}^{(t)} = \frac{1}{1 - \rho_2^t} \mathbf{r}^{(t)}$
 - update:
 - * $\mathbf{w}^{(t)} = \mathbf{w}^{(t-1)} - \frac{\eta}{\delta + \sqrt{\hat{\mathbf{r}}^{(t)}}} \hat{\mathbf{s}}^{(t)}$

22 Example

- change the optimizer when compiling the network.

```
[28]: # Create model instance
model = DeepCNN()

# Move model to device (GPU if available)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
print(f"Using device: {device}")

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

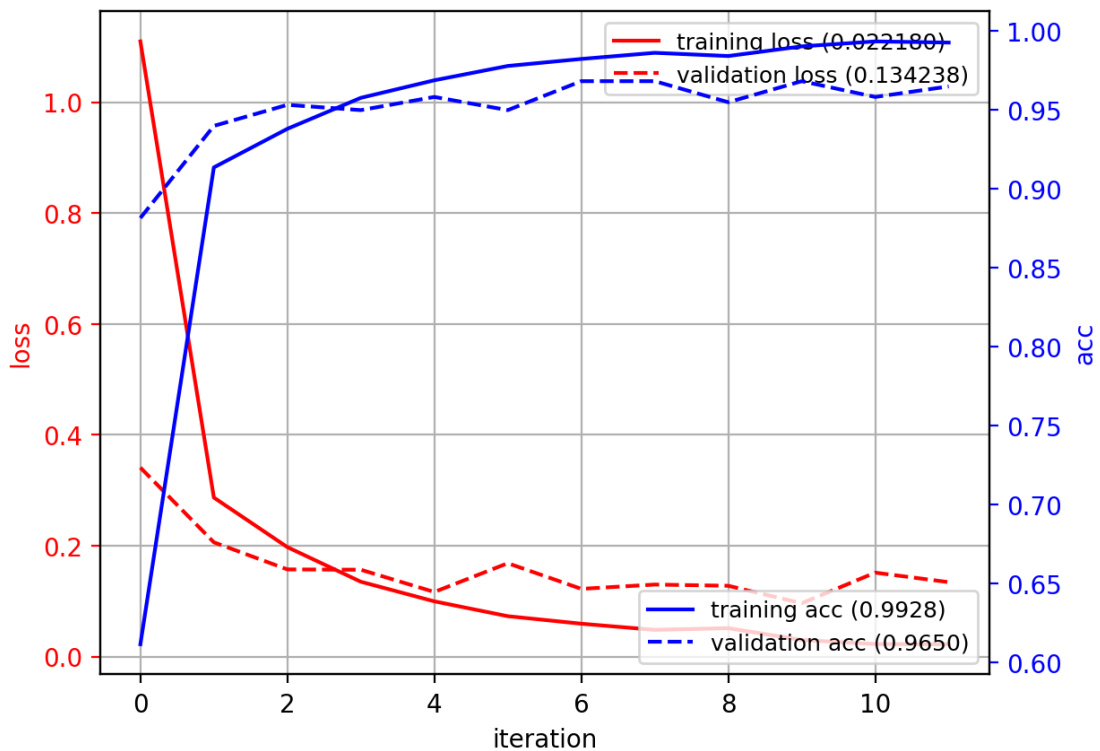
# Train the model
history, model = train_model(model, train_loader, valid_loader, criterion,
                               optimizer, epochs=100, patience=5)
```

```
Using device: cuda
Epoch 1/100:   Train Loss: 1.109883, Train Acc: 61.17%;   Val Loss: 0.340950,
Val Acc: 88.17%
Epoch 2/100:   Train Loss: 0.286864, Train Acc: 91.37%;   Val Loss: 0.206138,
Val Acc: 94.00%
Epoch 3/100:   Train Loss: 0.197446, Train Acc: 93.81%;   Val Loss: 0.157247,
Val Acc: 95.33%
Epoch 4/100:   Train Loss: 0.134986, Train Acc: 95.78%;   Val Loss: 0.156642,
Val Acc: 95.00%
EarlyStopping counter: 1 out of 5
Epoch 5/100:   Train Loss: 0.099617, Train Acc: 96.89%;   Val Loss: 0.116509,
Val Acc: 95.83%
Epoch 6/100:   Train Loss: 0.072853, Train Acc: 97.80%;   Val Loss: 0.168416,
Val Acc: 95.00%
EarlyStopping counter: 1 out of 5
Epoch 7/100:   Train Loss: 0.059275, Train Acc: 98.24%;   Val Loss: 0.122134,
Val Acc: 96.83%
Epoch 8/100:   Train Loss: 0.048134, Train Acc: 98.63%;   Val Loss: 0.129905,
Val Acc: 96.83%
EarlyStopping counter: 1 out of 5
Epoch 9/100:   Train Loss: 0.051222, Train Acc: 98.43%;   Val Loss: 0.127682,
Val Acc: 95.50%
EarlyStopping counter: 2 out of 5
Epoch 10/100:  Train Loss: 0.029914, Train Acc: 99.04%;   Val Loss: 0.096202,
Val Acc: 96.83%
EarlyStopping counter: 3 out of 5
Epoch 11/100:  Train Loss: 0.022572, Train Acc: 99.35%;   Val Loss: 0.151418,
Val Acc: 95.83%
EarlyStopping counter: 4 out of 5
```


Epoch 12/100: Train Loss: 0.022180, Train Acc: 99.28%; Val Loss: 0.134238, Val Acc: 96.50%
EarlyStopping counter: 5 out of 5
Early stopping triggered

```
[29]: # Plot the training history
plot_history(history)

acc = evaluate_model(model, test_loader)
print("test accuracy:", acc)
```



test accuracy: 96.52

23 Which optimizer is best?

- there's no best optimizer...
- based on the problem and own familiarity with tuning the hyperparameters.